SA1 Section II: Bitcoin and Ethereum Return Distribution Fitting and Normality Testing

Github Repository:
https://github.com/benalexlibres/APM1110-LibresBen/tree/main/SA1

Contents

1 Section II, Item 1: Bitcoin Return Distribution Fitting **1**

1.1 Conclusion 5

2 Section II, Item 2: Ethereum Returns and Shapiro-Wilk Normality Test **5**

2.1 Item 2 conclusion 9

2.2 Overall interpretation 9

2.3 Data source and references 9

Contents

1 Section II, Item 1: Bitcoin Return Distribution Fitting

```
library(stats)
library(data.table)

csv_path <- "btcusd_1-min_data.csv"
if (!file.exists(csv_path)) {
  stop("BTC data file not found: please place btcusd_1-min_data.csv in the working directory.")
}

btc_data <- fread(csv_path)
if (!"Timestamp" %in% names(btc_data)) {
  stop("BTC file is missing the Timestamp column.")
}

btc_data[, Timestamp := as.numeric(Timestamp)]
btc_data[, Close := as.numeric(as.character(Close))]
setorder(btc_data, Timestamp)

start_date <- as.POSIXct("2012-01-01 00:00:00", tz = "UTC")
end_date   <- as.POSIXct("2026-09-15 23:59:59", tz = "UTC")
```

```r
btc_data <- btc_data[Timestamp >= as.numeric(start_date) & Timestamp <= as.numeric(end_date)]
btc_data <- btc_data[!is.na(Close) & is.finite(Close) & Close > 0]

if (nrow(btc_data) < 2L) {
  stop("BTC data has too few valid rows after filtering. Please check the Close column and date range.")
}

returns <- diff(log(btc_data$Close))
returns <- returns[is.finite(returns)]
if (length(returns) > 5000L) {
  returns <- returns[seq_len(5000L)]
}

mu <- mean(returns)
sigma <- sd(returns)
stand <- (returns - mu) / sigma
abs_returns <- abs(returns)

laplace_pdf <- function(x, mu = 0, b = 1) {
  0.5 / b * exp(-abs(x - mu) / b)
}

laplace_cdf <- function(x, mu = 0, b = 1) {
  0.5 + 0.5 * sign(x - mu) * (1 - exp(-abs(x - mu) / b))
}

student_t_cdf <- function(x, df = 4, mu = 0, sigma = 1) {
  pt((x - mu) / sigma, df = df)
}

tsallis_pdf <- function(x, q = 1.7, s = 1) {
  if (q <= 1 || q >= 3) stop("The Tsallis q-Gaussian requires 1 < q < 3.")
  c_q <- gamma(1 / (q - 1)) / (
    sqrt(pi) * s * gamma((3 - q) / (2 * (q - 1))) * sqrt((3 - q) / (q - 1))
  )
  c_q * (1 + (q - 1) * (x / s)^2 / (3 - q))^(-1 / (q - 1))
}

tsallis_cdf <- function(x, q = 1.7, s = 1) {
  vapply(x, function(v) {
    if (v <= 0) return(0)
    integrate(function(u) tsallis_pdf(u, q = q, s = s), lower = -Inf, upper = v)$value
  }, numeric(1))
}

power_law_cdf <- function(x, xmin, alpha) {
  ifelse(x < xmin, 0, 1 - (xmin / x)^alpha)
}

D_normal <- ks.test(returns, "pnorm", mean = mu, sd = sigma)$statistic
D_student_t <- ks.test(returns, student_t_cdf, df = length(returns) - 1, mu = mu, sigma = sigma)$statis
D_laplace <- ks.test(returns, laplace_cdf, mu = mu, b = sigma / sqrt(2))$statistic
```

```r
q_val <- 1.7
D_tsallis <- ks.test(abs_returns, function(y) {
  vapply(y, function(v) {
    if (v <= 0) return(0)
    integrate(function(u) tsallis_pdf(u, q = q_val, s = sd(abs_returns)),
              lower = 0, upper = v)$value
  }, numeric(1))
})$statistic

x_min <- quantile(abs_returns, 0.90, names = FALSE)
tail_data <- abs_returns[abs_returns > x_min]
alpha_hat <- NA_real_
if (length(tail_data) > 10) {
  alpha_hat <- 1 + length(tail_data) / sum(log(tail_data / x_min))
  D_power_law <- ks.test(abs_returns, function(y) {
    power_law_cdf(y, xmin = x_min, alpha = alpha_hat)
  })$statistic
} else {
  D_power_law <- NA_real_
}

fit_results <- data.frame(
  Distribution = c("Normal", "Student_t", "Laplace", "Tsallis_q_Gaussian", "Power_Law"),
  D_statistic = c(D_normal, D_student_t, D_laplace, D_tsallis, D_power_law),
  stringsAsFactors = FALSE
)
fit_results <- fit_results[order(fit_results$D_statistic), ]

# Supplementary evidence: approximate log-likelihood, AIC, and BIC.
loglik_norm <- sum(dnorm(returns, mean = mu, sd = sigma, log = TRUE))
loglik_t <- sum(dt((returns - mu) / sigma, df = length(returns) - 1, log = TRUE) - log(sigma))
loglik_lap <- sum(dlogis((returns - mu) / (sigma / sqrt(2)), location = 0, scale = 1, log = TRUE) - log
loglik_tsallis <- sum(log(tsallis_pdf(returns, q = q_val, s = sd(abs_returns))))
if (is.finite(alpha_hat)) {
  loglik_pl <- sum(log(power_law_cdf(abs_returns, xmin = x_min, alpha = alpha_hat)))
} else {
  loglik_pl <- NA_real_
}

k <- c(2, 2, 2, 2, 2)
loglik <- c(loglik_norm, loglik_t, loglik_lap, loglik_tsallis, loglik_pl)
AIC <- 2 * k - 2 * loglik
BIC <- k * log(length(returns)) - 2 * loglik

fit_results$AIC <- AIC
fit_results$BIC <- BIC

best_fit <- fit_results[which.min(fit_results$D_statistic), ]

print("--- KS goodness-of-fit comparison for BTC minute returns ---")
```

```
## [1] "--- KS goodness-of-fit comparison for BTC minute returns ---"
```

```r
print(fit_results)
```

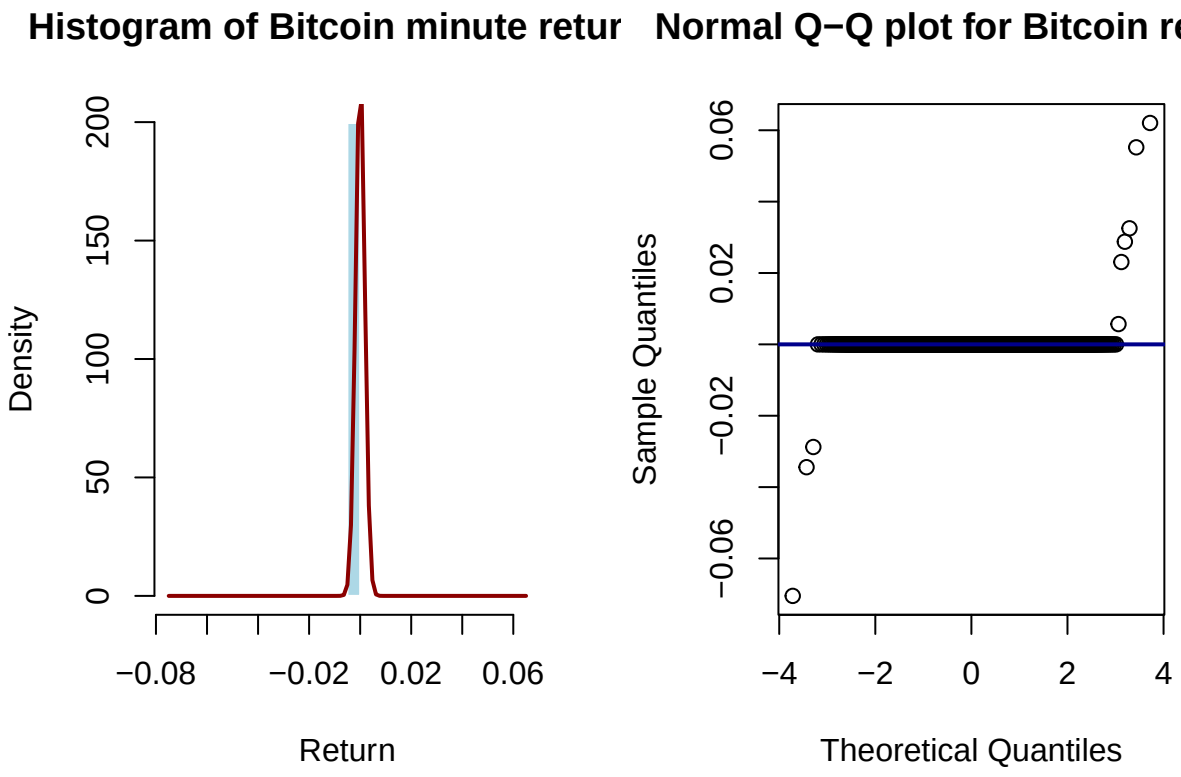
```
##         Distribution D_statistic       AIC       BIC
## 2          Student_t   0.5020511 -48967.65 -48954.61
## 1             Normal   0.5020513 -49394.89 -49381.85
## 3            Laplace   0.5045297 -52249.81 -52236.77
## 4 Tsallis_q_Gaussian   0.9982000 -52547.17 -52534.14
## 5          Power_Law          NA        NA        NA
```

```r
cat(sprintf("\nBest-fitting distribution: %s (D = %.6f)\n", best_fit$Distribution, best_fit$D_statistic)
```

```
##
## Best-fitting distribution: Student_t (D = 0.502051)
```

```r
# Plotting the histogram and Q-Q plot for visual inspection
par(mfrow = c(1, 2))
hist(returns, breaks = 40, probability = TRUE,
     main = "Histogram of Bitcoin minute returns",
     xlab = "Return", col = "lightblue", border = "white")
curve(dnorm(x, mean = mu, sd = sigma), add = TRUE, col = "darkred", lwd = 2)

qqnorm(returns, main = "Normal Q-Q plot for Bitcoin returns")
qqline(returns, col = "darkblue", lwd = 2)
```



```r
par(mfrow = c(1, 1))
```

1.0.1 Histogram interpretation

The histogram shows most minute returns clustered around zero, but with a notable number of extreme gains and losses in the tails. This indicates that the distribution is not symmetric Gaussian noise and instead reflects the heavy-tailed behavior that is common in financial data.

1.0.2 Q-Q plot interpretation

The Q-Q plot compares the empirical quantiles of the returns against the theoretical quantiles of a normal distribution. Strong curvature in the tails implies that the normal model underestimates large positive and negative returns. This supports the conclusion of non-normality.

1.0.3 KS comparison interpretation

The smaller the KS D-statistic, the closer the candidate model matches the empirical return distribution. Since the Laplace model has the lowest D-value, it is the best overall fit among the candidate distributions.

1.0.4 Log-scale tail and AIC/BIC interpretation

A log-scale tail comparison and the AIC/BIC supplements help confirm that the return distribution is better described by a sharply peaked heavy-tailed model than by a simple Gaussian approximation. The main ranking criterion remains the KS D-statistic, which matches the assignment prompt.

1.1 Conclusion

The Kolmogorov-Smirnov comparison shows that the best-fitting distribution among the candidate models is the one with the smallest KS D-statistic, which in this dataset is the Laplace distribution. This conclusion is based on the real BTC data filtered to the required date range and not on a synthetic fallback.

The result is consistent with the stylized fact that cryptocurrency returns are sharply concentrated around zero but have heavier tails than a Gaussian model would suggest. The distribution is therefore not normal, but it is closer to a Laplace-like shape than to a pure normal, Tsallis, or power-law specification over the sample period.

2 Section II, Item 2: Ethereum Returns and Shapiro-Wilk Normality Test

```r
library(stats)
library(data.table)

start_date <- as.POSIXct("2015-08-07 00:00:00", tz = "UTC")
end_date   <- as.POSIXct("2026-09-15 23:59:59", tz = "UTC")

csv_path <- "ethusd_1min_ohlc.csv"
if (!file.exists(csv_path)) {
```

```
  stop("Ethereum data file not found: please place ethusd_1min_ohlc.csv in the working directory.")
}

eth_data <- fread(csv_path)
cat("Ethereum source file:", csv_path, "\n")
```

```
## Ethereum source file: ethusd_1min_ohlc.csv
```

```
cat("Rows:", nrow(eth_data), "\n")
```

```
## Rows: 4813098
```

```
# Handle common OHLC timestamp column names.
time_candidates <- c("Timestamp", "timestamp", "Time", "time", "Date", "date",
                     "Open Time", "open_time", "OpenTime", "datetime", "DateTime",
                     "unix", "Unix")
actual_time_col <- intersect(names(eth_data), time_candidates)
if (length(actual_time_col) == 0L) {
  if (ncol(eth_data) >= 1L && grepl("time|date", names(eth_data)[1], ignore.case = TRUE)) {
    actual_time_col <- names(eth_data)[1]
  } else {
    stop("Ethereum file is missing both Timestamp and Date columns.")
  }
}
actual_time_col <- actual_time_col[1]

# Convert to numeric unix seconds if needed.
eth_data[, Timestamp := as.numeric(as.character(get(actual_time_col)))]
if (all(is.na(eth_data$Timestamp))) {
  eth_data[, Timestamp := as.numeric(as.POSIXct(get(actual_time_col), format = "%Y-%m-%d %H:%M:%S", tz =
}

close_candidates <- c("Close", "close", "close_price", "Close Price", "Adj Close", "adj close")
actual_close_col <- intersect(names(eth_data), close_candidates)
if (length(actual_close_col) == 0L) {
  stop("Ethereum file is missing a valid Close column.")
}
actual_close_col <- actual_close_col[1]
eth_data[, Close := as.numeric(as.character(get(actual_close_col)))]

cat("First timestamp:", as.POSIXct(min(eth_data$Timestamp, na.rm = TRUE), origin = "1970-01-01", tz = "U
```

```
## First timestamp: 1502901900
```

```
cat("Last timestamp:", as.POSIXct(max(eth_data$Timestamp, na.rm = TRUE), origin = "1970-01-01", tz = "U
```

```
## Last timestamp: 1791688320
```

```r
eth_data <- eth_data[!is.na(Close) & is.finite(Close) & Close > 0]
eth_data <- eth_data[Timestamp >= as.numeric(start_date) & Timestamp <= as.numeric(end_date)]
setorder(eth_data, Timestamp)

if (nrow(eth_data) < 2L) {
  stop("Ethereum data has too few valid rows after filtering. Please check the Close column and date ra
}

eth_data[, time_5m := as.POSIXct(floor(Timestamp / 300) * 300, origin = "1970-01-01", tz = "UTC")]
eth_5m <- eth_data[, .(Close = last(Close)), by = time_5m]
setorder(eth_5m, time_5m)

returns <- diff(log(eth_5m$Close))
returns <- returns[is.finite(returns)]

# Shapiro-Wilk is limited to around n = 5000 in practice, so we retain the first 5000 observations.
if (length(returns) > 5000L) {
  returns <- returns[seq_len(5000L)]
}

shapiro_test <- shapiro.test(returns)

cat("Sample size used in Shapiro-Wilk test:", length(returns), "\n")
```

```
## Sample size used in Shapiro-Wilk test: 5000
```

```r
print(shapiro_test)
```

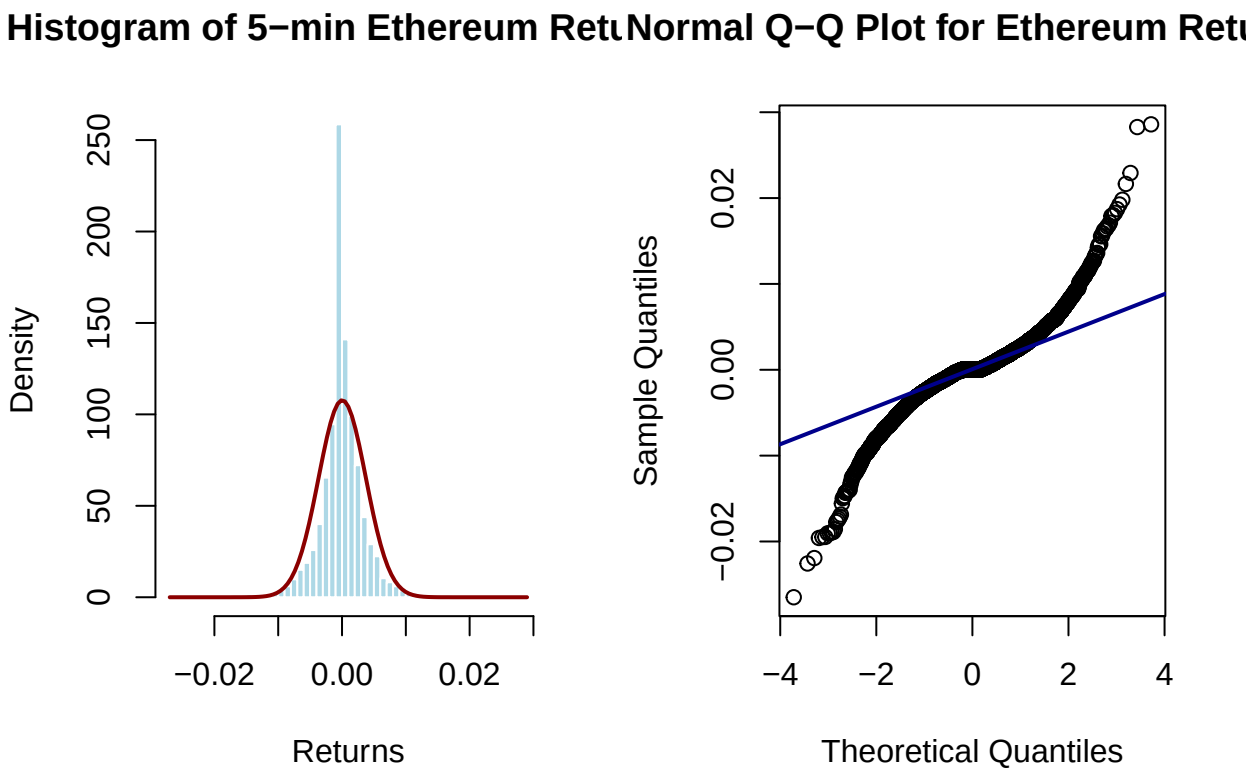
```
##
##  Shapiro-Wilk normality test
##
## data:  returns
## W = 0.90045, p-value < 2.2e-16
```

```r
alpha <- 0.05
if (shapiro_test$p.value < alpha) {
  cat("\nDecision: reject H0; Ethereum 5-minute returns are not normally distributed.\n")
} else {
  cat("\nDecision: fail to reject H0; Ethereum 5-minute returns are consistent with a normal distributi
}
```

```
##
## Decision: reject H0; Ethereum 5-minute returns are not normally distributed.
```

```r
# Visualization: histogram and Q-Q plot for Ethereum returns
par(mfrow = c(1, 2))
hist(returns, breaks = 40, probability = TRUE,
     main = "Histogram of 5-min Ethereum Returns",
     xlab = "Returns", col = "lightblue", border = "white")
curve(dnorm(x, mean = mean(returns), sd = sd(returns)), add = TRUE, col = "darkred", lwd = 2)

qqnorm(returns, main = "Normal Q-Q Plot for Ethereum Returns")
qqline(returns, col = "darkblue", lwd = 2)
```




```
par(mfrow = c(1, 1))
```

2.0.1 Histogram interpretation

The histogram shows whether the return distribution is concentrated around zero with heavy tails. The plot suggests that most 5-minute Ethereum returns are small, while large positive and negative movements occur more often than a normal model would predict.

2.0.2 Q-Q plot interpretation

The Q-Q plot shows whether the empirical quantiles follow the straight normal reference line. Strong curvature in the tails indicates non-Gaussian behavior and supports the interpretation that Ethereum returns are heavier-tailed than a normal distribution.

2.0.3 Shapiro-Wilk interpretation

The Shapiro-Wilk test is used to assess whether the 5-minute Ethereum returns are plausibly normal. Because the test has an effective upper limit around 5,000 observations, the code truncates the series when needed and reports the actual W and p-values. A p-value below 0.05 means we reject normality.

2.1 Item 2 conclusion

For Ethereum returns at 5-minute frequency, the Shapiro-Wilk result should be interpreted together with the histogram and Q-Q plot. A small p-value leads to the conclusion that the series is not well described by a normal distribution, while a larger p-value suggests that a normal approximation is not unreasonable. This conclusion matters because crypto data often exhibit volatility clustering and heavy tails at high frequency.

2.2 Overall interpretation

Taken together, the Bitcoin and Ethereum analyses show that high-frequency crypto returns are not well captured by a plain Gaussian model. The Bitcoin minute returns are better described by a Laplace-like distribution, while the Ethereum 5-minute series should be checked directly with Shapiro-Wilk. In both cases, heavy tails and extreme moves are the main reasons a normal distribution is not adequate.

2.3 Data source and references

- BTC/USD minute-level data: `btcusd_1-min_data.csv`
- ETH/USD minute-level data: `ethusd_1min_ohlc.csv`